

BÁO CÁO MÃ NGUỒN - LAB 04

****Sinh viên thực hiện:**** Lê Viết Đức

****MSSV:**** 23DTHA4 / 0533

****Môn học:**** Thực hành Bảo mật thông tin nâng cao

Đường dẫn file: `lab-04\requirements.txt`

```
pycryptodome  
cryptography  
tornado
```

Đường dẫn file: `lab-04\aes\_rsa\_socket\client.py`

```
from Crypto.Cipher import AES, PKCS1_OAEP  
from Crypto.PublicKey import RSA  
from Crypto.Random import get_random_bytes  
from Crypto.Util.Padding import pad, unpad  
import socket  
import threading  
import hashlib  
import sys  
  
# Initialize client socket  
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
client_socket.connect(('localhost', 12345))  
  
# Generate RSA key pair  
client_key = RSA.generate(2048)  
  
# Receive server's public key  
server_public_key = RSA.import_key(client_socket.recv(2048))  
  
# Send client's public key to the server  
client_socket.send(client_key.publickey().export_key(format='PEM'))  
  
# Receive encrypted AES key from the server  
encrypted_aes_key = client_socket.recv(2048)
```

```

# Decrypt the AES key using client's private key
cipher_rsa = PKCS1_OAEP.new(client_key)
aes_key = cipher_rsa.decrypt(encrypted_aes_key)

# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to receive messages from server
def receive_messages():
    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        sys.stdout.write('\nReceived: ' + decrypted_message + '\n') # In tin nhắn đã nhận
        sys.stdout.write("Enter message ('exit' to quit): ") # In lại prompt nhập tin nhắn
        sys.stdout.flush()

# Start the receiving thread
receive_thread = threading.Thread(target=receive_messages)
receive_thread.start()

# Send messages to the server
while True:
    sys.stdout.write("Enter message ('exit' to quit): ")
    sys.stdout.flush() # Đảm bảo prompt được in ngay lập tức
    message = input() # Nhận đầu vào từ người dùng
    encrypted_message = encrypt_message(aes_key, message)
    client_socket.send(encrypted_message)
    if message == "exit":
        break

# Close the connection when done
client_socket.close()

```

Đường dẫn file: `lab-04\aes\_rsa\_socket\client2.py`

```

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import socket
import threading
import hashlib
import sys

# Initialize client socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))

# Generate RSA key pair
client_key = RSA.generate(2048)

# Receive server's public key
server_public_key = RSA.import_key(client_socket.recv(2048))

# Send client's public key to the server
client_socket.send(client_key.publickey().export_key(format='PEM'))

# Receive encrypted AES key from the server
encrypted_aes_key = client_socket.recv(2048)

# Decrypt the AES key using client's private key
cipher_rsa = PKCS1_OAEP.new(client_key)
aes_key = cipher_rsa.decrypt(encrypted_aes_key)

# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to receive messages from server
def receive_messages():
    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        sys.stdout.write('\nReceived: ' + decrypted_message + '\n') # In tin nhắn đã nhận
        sys.stdout.write("Enter message ('exit' to quit): ") # In lại prompt nhập tin nhắn

```

```

        sys.stdout.flush()

# Start the receiving thread
receive_thread = threading.Thread(target=receive_messages)
receive_thread.start()

# Send messages to the server
while True:
    sys.stdout.write("Enter message ('exit' to quit): ")
    sys.stdout.flush() # Đảm bảo prompt được in ngay lập tức
    message = input() # Nhận đầu vào từ người dùng
    encrypted_message = encrypt_message(aes_key, message)
    client_socket.send(encrypted_message)
    if message == "exit":
        break

# Close the connection when done
client_socket.close()

```

Đường dẫn file: `lab-04\aes\_rsa\_socket\requirements.txt`

```

pycryptodome

```

Đường dẫn file: `lab-04\aes\_rsa\_socket\server.py`

```

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import socket
import threading
import hashlib

# Initialize server socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 12345))
server_socket.listen(5)

# Generate RSA key pair
server_key = RSA.generate(2048)

# List of connected clients

```

```

clients = []

# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to handle client connection
def handle_client(client_socket, client_address):
    print(f"Connected with {client_address}")

    # Send server's public key to client
    client_socket.send(server_key.publickey().export_key(format='PEM'))

    # Receive client's public key
    client_received_key = RSA.import_key(client_socket.recv(2048))

    # Generate AES key for message encryption
    aes_key = get_random_bytes(16)

    # Encrypt the AES key using the client's public key
    cipher_rsa = PKCS1_OAEP.new(client_received_key)
    encrypted_aes_key = cipher_rsa.encrypt(aes_key)
    client_socket.send(encrypted_aes_key)

    # Add client to the list
    clients.append((client_socket, aes_key))

    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        print(f"Received from {client_address}: {decrypted_message}")

        # Send received message to all other clients
        for client, key in clients:
            if client != client_socket:
                encrypted = encrypt_message(key, decrypted_message)
                client.send(encrypted)

        if decrypted_message == "exit":
            break

```

```

clients.remove((client_socket, aes_key))
client_socket.close()
print(f"Connection with {client_address} closed")

# Accept and handle client connections
while True:
    client_socket, client_address = server_socket.accept()
    client_thread = threading.Thread(target=handle_client, args=(client_socket, client_address))
    client_thread.start()

```

Đường dẫn file: `lab-04\dh\_key\_pair\client.py`

```

from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

def generate_client_key_pair(parameters):
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key

def derive_shared_secret(private_key, server_public_key):
    shared_key = private_key.exchange(server_public_key)
    return shared_key

def main():
    # Load server's public key
    with open("server_public_key.pem", "rb") as f:
        server_public_key = serialization.load_pem_public_key(f.read())

    parameters = server_public_key.parameters()
    private_key, public_key = generate_client_key_pair(parameters)

    shared_secret = derive_shared_secret(private_key, server_public_key)

    print("Shared Secret:", shared_secret.hex())

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\dh\_key\_pair\requirements.txt`

cryptography

Đường dẫn file: `lab-04\dh\_key\_pair\server.py`

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization

def generate_dh_parameters():
    parameters = dh.generate_parameters(generator=2, key_size=2048)
    return parameters

def generate_server_key_pair(parameters):
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key

def main():
    parameters = generate_dh_parameters()
    private_key, public_key = generate_server_key_pair(parameters)

    with open("server_public_key.pem", "wb") as f:
        f.write(public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

if __name__ == "__main__":
    main()
```

Đường dẫn file: `lab-04\hash\blake2\_hash.py`

```
import hashlib

def blake2(message):
    blake2_hash = hashlib.blake2b(digest_size=64)
    blake2_hash.update(message)
    return blake2_hash.digest()

def main():
    text = input("Nhập chuỗi văn bản: ").encode('utf-8')
    hashed_text = blake2(text)
```

```

print("Chuỗi văn bản đã nhập:", text.decode('utf-8'))
print("BLAKE2 Hash:", hashed_text.hex())

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\hash\md5\_hash.py`

```

# Hàm dịch vòng trái (left rotate) cho 1 giá trị 32-bit
def left_rotate(value, shift):
    return ((value << shift) | (value >> (32 - shift))) & 0xFFFFFFFF

# Hàm băm MD5 chính
def md5(message):
    # Khởi tạo các biến ban đầu theo chuẩn MD5
    a = 0x67452301
    b = 0xEFCDAB89
    c = 0x98BADCFE
    d = 0x10325476

    # Tiền xử lý chuỗi văn bản (padding)
    original_length = len(message)
    message += b'\x80' # Thêm bit '1' vào cuối chuỗi
    while len(message) % 64 != 56: # Đệm thêm các bit '0' sao cho độ dài mod 64 == 56
        message += b'\x00'
    message += original_length.to_bytes(8, 'little') # Thêm độ dài ban đầu (64-bit little
    endian)

    # Chia chuỗi thành các khối 512-bit (64 bytes)
    for i in range(0, len(message), 64):
        block = message[i:i+64]

        # Tách block thành 16 từ 32-bit (little endian)
        words = [int.from_bytes(block[j:j+4], 'little') for j in range(0, 64, 4)]

        # Lưu giá trị ban đầu
        a0, b0, c0, d0 = a, b, c, d

        # Vòng lặp chính của thuật toán MD5 gồm 64 bước
        for j in range(64):
            if j < 16:
                f = (b & c) | ((~b) & d)
                g = j
            elif j < 32:

```



```

        f = (d & b) | ((~d) & c)
        g = (5 * j + 1) % 16
    elif j < 48:
        f = b ^ c ^ d
        g = (3 * j + 5) % 16
    else:
        f = c ^ (b | (~d))
        g = (7 * j) % 16

    temp = d
    d = c
    c = b
    # Thực hiện các phép tính chính: cộng, dịch vòng trái, cộng thêm giá trị trong
words[g]

    b = (b + left_rotate((a + f + 0x5A827999 + words[g]) & 0xFFFFFFFF, 3)) & 0xFFFFFFFF
    a = temp

    # Cộng dồn vào giá trị ban đầu
    a = (a + a0) & 0xFFFFFFFF
    b = (b + b0) & 0xFFFFFFFF
    c = (c + c0) & 0xFFFFFFFF
    d = (d + d0) & 0xFFFFFFFF

    # Trả về chuỗi băm ở dạng hex
    return '{:08x}{:08x}{:08x}{:08x}'.format(a, b, c, d)

# Nhập dữ liệu từ người dùng
input_string = input("Nhập chuỗi cần băm: ")
md5_hash = md5(input_string.encode('utf-8'))

# In kết quả
print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))

```

Đường dẫn file: `lab-04\hash\md5\_hash\_2.py`

```

import math
# Hàm dịch vòng trái (left rotate) cho 1 giá trị 32-bit
def left_rotate(value, shift):
    return ((value << shift) | (value >> (32 - shift))) & 0xFFFFFFFF

# Bảng hằng số K dùng trong MD5 (64 phần tử)
K = [int((2**32) * abs(math.sin(i + 1))) & 0xFFFFFFFF for i in range(64)]

# Bảng số lượng dịch trái (shift amounts)
s = [

```

```

7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,
5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,
4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,
6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21
]

# Hàm băm MD5 chính
def md5(message):
    # Khởi tạo các giá trị ban đầu (IV)
    a0 = 0x67452301
    b0 = 0xefcdab89
    c0 = 0x98badcfe
    d0 = 0x10325476

    # Tiền xử lý chuỗi (Padding)
    original_length = len(message) * 8 # Tính độ dài bit
    message += b'\x80'
    while (len(message) % 64) != 56:
        message += b'\x00'
    message += original_length.to_bytes(8, byteorder='little') # Thêm độ dài (little endian)

    # Xử lý các khối 512-bit (64 byte)
    for offset in range(0, len(message), 64):
        block = message[offset:offset + 64]
        M = [int.from_bytes(block[i:i + 4], byteorder='little') for i in range(0, 64, 4)]

        A, B, C, D = a0, b0, c0, d0 # Sao chép giá trị ban đầu

        # Vòng lặp chính (64 bước)
        for i in range(64):
            if i < 16:
                F = (B & C) | (~B & D)
                g = i
            elif i < 32:
                F = (D & B) | (~D & C)
                g = (5 * i + 1) % 16
            elif i < 48:
                F = B ^ C ^ D
                g = (3 * i + 5) % 16
            else:
                F = C ^ (B | ~D)
                g = (7 * i) % 16

            temp = (A + F + K[i] + M[g]) & 0xFFFFFFFF
            A, D, C, B = D, C, B, (B + left_rotate(temp, s[i])) & 0xFFFFFFFF

        # Cộng dồn vào kết quả
        a0 = (a0 + A) & 0xFFFFFFFF
        b0 = (b0 + B) & 0xFFFFFFFF
        c0 = (c0 + C) & 0xFFFFFFFF
        d0 = (d0 + D) & 0xFFFFFFFF

```

```
# Trả về kết quả dạng hex (little endian)
digest = sum(x << (32 * i) for i, x in enumerate([a0, b0, c0, d0]))
return ''.join(f'{digest >> (8 * i) & 0xFF:02x}' for i in range(16))

# ===== Chạy thử =====
input_string = input("Nhập chuỗi cần băm: ")
md5_hash = md5(input_string.encode('utf-8'))

print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))
```

Đường dẫn file: `lab-04\hash\md5\_library.py`

```
import hashlib

def calculate_md5(input_string):
    md5_hash = hashlib.md5()
    md5_hash.update(input_string.encode('utf-8'))
    return md5_hash.hexdigest()

input_string = input("Nhập chuỗi cần băm: ")

md5_hash = calculate_md5(input_string)

# In ra kết quả mã băm
print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))
```

Đường dẫn file: `lab-04\hash\sha256\_hash.py`

```
import hashlib

def calculate_sha256_hash(data):
    sha256_hash = hashlib.sha256()
    sha256_hash.update(data.encode('utf-8'))
    return sha256_hash.hexdigest()

data_to_hash = input("Nhập dữ liệu để hash bằng SHA-256: ")
hash_value = calculate_sha256_hash(data_to_hash)
print("Giá trị hash SHA-256:", hash_value)
```

Đường dẫn file: `lab-04\hash\sha3\_hash.py`

```

from Crypto.Hash import SHA3_256

def sha3(message):
    sha3_hash = SHA3_256.new()
    sha3_hash.update(message)
    return sha3_hash.digest()

def main():
    text = input("Nhập chuỗi văn bản: ").encode('utf-8')
    hashed_text = sha3(text)

    print("Chuỗi văn bản đã nhập:", text.decode('utf-8'))
    print("SHA-3 Hash:", hashed_text.hex())

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\websocket\client.py`

```

import tornado.ioloop
import tornado.websocket

class WebSocketClient:
    def __init__(self, io_loop):
        self.connection = None
        self.io_loop = io_loop

    def start(self):
        self.connect_and_read()

    def stop(self):
        self.io_loop.stop()

    def connect_and_read(self):
        print("Reading...")
        tornado.websocket.websocket_connect(
            url="ws://localhost:8888/websocket/",
            callback=self.maybe_retry_connection,
            on_message_callback=self.on_message,
            ping_interval=30,
            ping_timeout=10,
        )

```

```

def maybe_retry_connection(self, future) -> None:
    try:
        self.connection = future.result()
    except:
        print('Could not reconnect, retrying in 3 seconds...')
        self.io_loop.call_later(3, self.connect_and_read)
        return

    self.connection.read_message(callback=self.on_message)

def on_message(self, message):
    if message is None:
        print('Disconnected, reconnecting...')
        self.connect_and_read()
        return

    print(f"Received word from server: {message}")

    self.connection.read_message(callback=self.on_message)

def main():
    io_loop = tornado.ioloop.IOLoop.current()

    client = WebSocketClient(io_loop)
    io_loop.add_callback(client.start)

    io_loop.start()

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\websocket\client2.py`

```

import tornado.ioloop
import tornado.websocket

class WebSocketClient:
    def __init__(self, io_loop):
        self.connection = None
        self.io_loop = io_loop

    def start(self):
        self.connect_and_read()

    def stop(self):
        self.io_loop.stop()

```

```

def connect_and_read(self):
    print("Reading...")
    tornado.websocket.websocket_connect(
        url="ws://localhost:8888/websocket/",
        callback=self.maybe_retry_connection,
        on_message_callback=self.on_message,
        ping_interval=30,
        ping_timeout=10,
    )

def maybe_retry_connection(self, future) -> None:
    try:
        self.connection = future.result()
    except:
        print('Could not reconnect, retrying in 3 seconds...')
        self.io_loop.call_later(3, self.connect_and_read)
        return

    self.connection.read_message(callback=self.on_message)

def on_message(self, message):
    if message is None:
        print('Disconnected, reconnecting...')
        self.connect_and_read()
        return

    print(f"Received word from server: {message}")

    self.connection.read_message(callback=self.on_message)

def main():
    io_loop = tornado.ioloop.IOLoop.current()

    client = WebSocketClient(io_loop)
    io_loop.add_callback(client.start)

    io_loop.start()

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\websocket\requirements.txt`

```
tornado
```

Đường dẫn file: `lab-04\websocket\server.py`

```

import random
import tornado.web
import tornado.ioloop
import tornado.websocket

class WebSocketServer(tornado.websocket.WebSocketHandler):
    clients = set()

    def open(self):
        WebSocketServer.clients.add(self)

    def on_close(self):
        WebSocketServer.clients.remove(self)

    @classmethod
    def send_message(cls, message: str):
        print(f"Sending message to {len(cls.clients)} client(s).")
        for client in cls.clients:
            client.write_message(message)

class RandomWordSelector:
    def __init__(self, word_list):
        self.word_list = word_list

    def sample(self):
        return random.choice(self.word_list)

def main():
    app = tornado.web.Application([
        (r"/websocket/", WebSocketServer),
    ], websocket_ping_interval=30,
        websocket_ping_timeout=10)

    app.listen(8888)

    io_loop = tornado.ioloop.IOLoop.current()

    word_selector = RandomWordSelector(['apple', 'banana', 'orange', 'grape', 'melon'])

    periodic_callback = tornado.ioloop.PeriodicCallback(
        lambda: WebSocketServer.send_message(word_selector.sample()), 3000
    )
    periodic_callback.start()

    io_loop.start()

```

```
if __name__ == "__main__":  
    main()
```
